

Software Test Documentation

COVID-19 Cough Detection Web Application

Version 1.0

Prepared by: Drew Quashie, Amanda Ogbonna, Loleyi Oluwatomisin, Richard Alonso Garcia

Faculty Advisor: Dr. Zahra Nematzadeh

1 Introduction

1.1 Overview

This test plan defines procedures for verifying the functional and non-functional requirements of the COVID-19 Cough Detection web application. Testing ensures that all required features and behaviors are implemented, and that they function reliably under normal and abnormal conditions: from clean three-cough captures to corrupted uploads, non-cough audio, and network failures.

1.2 Purpose

The purpose of this plan is to establish systematic testing guidelines for each requirement documented in the Software Requirements Specification (`requirements_specification.md`). Systematic testing is documented for each requirement to confirm correct system behavior, identify failures, and validate compliance with user and technical expectations.

1.3 Approach

Each functional and non-functional requirement (SFR/SNFR) is translated into one or more test cases. Each test case specifies inputs, expected outputs, dependencies, and procedures. Test inputs include both typical and atypical values. Execution will cover unit testing (segmentation, voting, threshold logic), integration testing (React ↔ FastAPI ↔ ML layer), system testing (full capture-to-result flows in real browsers), and acceptance testing (scenarios UCS1–UCS4 walked through with end users).

1.4 References

- IEEE Std 829-2008 (Software Test Documentation Standard).
- `requirements_specification.pdf`: Software Requirements Specification, v1.0.
- `software_design_document.pdf`: Software Design Document, v1.0.
- CoughVid Dataset Documentation, ETH Zurich.

- Team Project Plan and Presentation documents.

1.5 Contents

The remainder of this document is structured as follows:

- Section 2: Test Case Scenarios for Software Functional Requirements.
 - Section 3: Test Case Scenarios for Software Non-Functional Requirements.
 - Section 4: Conclusion.
-

2 Test Case Scenarios for Software Functional Requirements

2.1 SFR1: Microphone permission and audio recording

- **Objective:** Verify audio capture through the Web Audio API with correct permission handling.
- **Inputs:** User opens the recorder and grants (or denies) the browser's microphone prompt.
- **Expected Outcome:** On grant, recording initializes and the input level meter responds to sound; on denial, a clear instructions prompt is shown instead of a broken screen.
- **Unusual Input:** Permission previously denied at browser level; no microphone hardware; multiple audio input devices present.
- **Dependencies:** Browser with `getUserMedia` support; user gesture required to start capture.
- **Procedure:** Open recorder on desktop and mobile browsers; accept and deny permission; confirm the meter moves only when a mic is live.

2.2 SFR2: Guided three-cough recording session

- **Objective:** Verify one capture session captures exactly three guided coughs with visible cues.
- **Inputs:** User starts recording and gives 3 coughs, one at a time.
- **Expected Outcome:** Cue dots fill ● ● ○ ● ● ● as each cough is detected; capture auto-stops after the third cough (manual stop also available); one clip is produced.
- **Unusual Input:** User coughs only once and stops; coughs four times; coughs over loud background speech; never coughs and lets the session run.
- **Dependencies:** SFR1 working; client-side onset detection active.
- **Procedure:** Run the happy path; then attempt early stop, extra coughs, and no-cough sessions; confirm the session state machine behaves as specified in all cases.

2.3 SFR3: Live level meter and client-side sanity checks

- **Objective:** Verify immediate feedback prevents obviously unusable recordings.
- **Inputs:** Quiet/silent input, normal cough, extremely loud/clipping input.

- **Expected Outcome:** Meter reflects input amplitude in real time; silence or gross clipping triggers a recapture prompt before upload.
- **Unusual Input:** Muted system mic; input pegged at max (clipping); background music only.
- **Dependencies:** SFR1.
- **Procedure:** Record in silence, at normal volume, and with a speaker held to the mic; confirm prompts appear only for unusable cases.

2.4 SFR4: Upload to `POST /v1/analyze`

- **Objective:** Verify single-request multipart upload of the recording.
- **Inputs:** Valid clip (5–15 s, <1 MB, WEBM or WAV).
- **Expected Outcome:** Request accepted; client shows the analyzing state; response JSON contains risk, confidence, vote, per_cough, quality/gate fields.
- **Unusual Input:** 2 MB file; 45 s clip; zero-byte file; unsupported codec.
- **Dependencies:** Backend reachable; endpoint versioned /v1.
- **Procedure:** Capture and upload a normal session; then replay oversized, over-duration, and empty payloads and confirm each is rejected by validation with a clear client message.

2.5 SFR5: Server input validation and quality gate

- **Objective:** Verify format/duration/size validation plus SNR/clipping gate fails fast with reasons.
- **Inputs:** Corrupted audio file; 0.3 s clip; silent recording; heavily clipped recording; valid recording.
- **Expected Outcome:** Invalid inputs return Inconclusive with the specific reason (e.g., "duration too short", "low SNR"); valid input passes to the cough gate. No 500 responses for bad user input.
- **Unusual Input:** Non-audio file renamed to .wav; NaN-duration metadata; valid audio in an exotic container.
- **Dependencies:** SFR4.
- **Procedure:** Send each malformed case directly to the endpoint; assert HTTP 200 Inconclusive-with-reason (per SDD: input problems are not server errors) and that ML work did not start (check latency logs).

2.6 SFR6: Cough-presence gate (XGBoost) before segmentation

- **Objective:** Verify non-cough audio is rejected before segmentation.
- **Inputs:** Music clip; speech recording; ambient noise; genuine cough recording.
- **Expected Outcome:** Non-cough inputs return Inconclusive("cough not detected") with no segmentation attempted; cough audio passes to segmentation.
- **Unusual Input:** Cough masked by loud background; whisper-quiet cough; coughing mixed with speech; human whistling shaped like a cough.
- **Dependencies:** Pretrained XGBoost detector artifact loaded at startup.

- **Procedure:** Submit a batch of non-cough and cough clips; assert gate verdicts; inspect pipeline logs to confirm segmenter ran only on gate passes.

2.7 SFR7: Segmentation into three 1 s windows

- **Objective:** Verify onset-centered fixed-length segmentation.
- **Inputs:** Valid three-cough recording passing the gate.
- **Expected Outcome:** Exactly three 1.0 s windows centered on detected cough events; short events padded with silence; all other audio discarded.
- **Unusual Input:** Only 2 detectable cough onsets; coughs closer together than 1 s; a cough at the very start/end of the clip.
- **Dependencies:** SFR6 pass; onset detector.
- **Procedure:** Unit-test the segmenter against synthetic clips with known onset positions; assert window count (3), length (1 s), and centering tolerance; assert <3 onsets returns Inconclusive(recapture) consistent with SFR5.

2.8 SFR8: Feature extraction (MFCC + log-Mel)

- **Objective:** Verify the joint feature contract per segment.
- **Inputs:** Three segmented 1 s windows.
- **Expected Outcome:** For each segment: log-Mel spectrogram and MFCCs computed, training-set normalization applied, stacked into the model's expected tensor shape.
- **Unusual Input:** All-silence segment (post-padding); segment at exactly 1.0 s boundary.
- **Dependencies:** Training-set normalization statistics available.
- **Procedure:** Unit-test shape and value ranges against a fixed reference clip; assert output tensors are deterministic run-to-run.

2.9 SFR9: CNN inference with 3-logit sigmoid output

- **Objective:** Verify per-cough classification: 3 logits $\rightarrow$ sigmoid $\rightarrow$ argmax.
- **Inputs:** Feature tensors from SFR8 (or reference tensors from a known checkpoint).
- **Expected Outcome:** Three independent sigmoid vectors returned; per-cough class = argmax; values in (0, 1), not required to sum to 1.
- **Unusual Input:** All three sigmoids below 0.5 (feeds confidence gate); identical logits across coughs.
- **Dependencies:** Model loaded and verified at startup.
- **Procedure:** Run inference on a golden clip and compare per-cough classes to expected values; assert the output head has exactly 3 logits.

2.10 SFR10: Deterministic vote aggregation

- **Objective:** Verify majority vote and the 1-1-1 tie-break are deterministic.
- **Inputs:** Crafted per-cough classes: [Pos, Pos, Neg]; [Pos, Unc, Neg] with known sigmoid vectors; repeated runs of the same input.
- **Expected Outcome:** [Pos, Pos, Neg] $\rightarrow$ Positive (majority); 1-1-1 $\rightarrow$ argmax of mean sigmoid vector; identical inputs always produce identical results.

- **Unusual Input:** Three-way split; two coughs producing identical logits; NaN in a logit (must be rejected upstream).
- **Dependencies:** SFR9.
- **Procedure:** Table-driven unit tests over all vote combinations (2-1 splits $\times$ 3 classes, plus 1-1-1 with each class winning the tie-break); run the same input 100 $\times$ and assert bit-identical output.

2.11 SFR11: Confidence gate and percentage

- **Objective:** Verify the <0.50 confidence gate and percentage display.
- **Inputs:** Crafted outputs with mean winning-class sigmoid 0.71 (pass) and 0.44 (fail); normal analysis runs.
- **Expected Outcome:** $\geq 0.50 \rightarrow$ result shown with confidence as a percentage; $<0.50 \rightarrow$ Inconclusive("low model confidence").
- **Unusual Input:** Confidence exactly 0.50 (boundary, expected to pass); confidence 0.49.
- **Dependencies:** SFR10.
- **Procedure:** Unit-test the boundary; then verify the UI renders the percentage (e.g., "Confidence: 62%") on passing results.

2.12 SFR12: Result display (risk, vote, signals, quality)

- **Objective:** Verify the result screen shows all required fields consistently.
- **Inputs:** Results for each risk level, crafted from known sigmoid vectors.
- **Expected Outcome:** Screen shows risk level, confidence, vote breakdown (winner + per-cough classes), mean signal strengths, and quality line ("cough detector pass $\cdot$ SNR good"); displayed risk matches the SDD mapping (Neg $\rightarrow$ LOW, Unc $\rightarrow$ MEDIUM, Pos with mean $\geq 0.65 \rightarrow$ HIGH else MEDIUM).
- **Unusual Input:** Vote winner Positive but mean Positive signal just below 0.65 (must show MEDIUM, never HIGH); quality line showing a gate/quality note.
- **Dependencies:** SFR10, SFR11.
- **Procedure:** Drive the UI with fixed server responses covering the mapping table; assert risk label, vote line, and signal values render exactly and never contradict each other.

2.13 SFR13: Disclaimers and recommendations

- **Objective:** Verify ethical framing on every result.
- **Inputs:** Results at each risk level; inconclusive results.
- **Expected Outcome:** Every result screen shows the pre-screening disclaimer; recommendation text matches risk (Low: monitor; Medium: test + mask; High: test soon + contact provider).
- **Unusual Input:** User navigates directly to a saved historical result.
- **Dependencies:** None.
- **Procedure:** Run 10+ analyses across levels; assert disclaimer presence on all, including history views and the inconclusive state.

2.14 SFR14: Local persistence and history management

- **Objective:** Verify IndexedDB storage, history list, and delete-all.
- **Inputs:** Multiple saved analyses; delete-all action.
- **Expected Outcome:** Records persist across reload/browser restart; dashboard lists them newest-first with risk and confidence; delete-all empties storage; no data appears in any network request after analysis.
- **Unusual Input:** Storage quota exceeded; private-browsing IndexedDB denial.
- **Dependencies:** SFR12 save action.
- **Procedure:** Save 3 results, reload, confirm all 3 shown; check DevTools → IndexedDB analyses store; run delete-all and confirm empty; repeat in a private window and confirm graceful degradation.

2.15 SFR15: Trends dashboard and export

- **Objective:** Verify longitudinal visualization and export formats.
- **Inputs:** ≥ 5 saved analyses spanning multiple days (seeded via SFR14).
- **Expected Outcome:** Trend chart plots risk and confidence over time; activity summary counts render; CSV export parses with expected columns; JSON export parses as valid record array.
- **Unusual Input:** Zero records (empty state message); one record; records with all three risk levels.
- **Dependencies:** SFR14.
- **Procedure:** Seed history, open Trends, verify chart and counts; export both formats and validate with a CSV parser and `JSON.parse`; test the empty state.

2.16 SFR16: UI states and responsive layout

- **Objective:** Verify explicit idle → recording → analyzing → result/inconclusive transitions on all screen sizes.
- **Inputs:** Normal analysis flow at phone and desktop widths.
- **Expected Outcome:** Each state renders distinctly (spinner labeled "Analyzing...", result screen, inconclusive screen with [Try Again]); no state shows stale data from a previous run.
- **Unusual Input:** Rapidly double-clicking record; navigating away mid-analysis; rescaling the window mid-flow.
- **Dependencies:** All prior SFRs.
- **Procedure:** Walk the full flow on a phone-sized viewport and desktop; interrupt the flow at each state; assert no crashes, no stuck spinners, no mixed states.

3 Test Case Scenarios for Software Non-Functional Requirements

3.1 SNFR1: Capture-to-result under 6 s; server inference under 1.5 s

- **Inputs:** Valid three-cough capture over a typical broadband connection.
- **Expected Outcome:** Server-side time (gate + 3 segmentation/inference passes) < 1.5 s; capture-to-displayed-result < 6 s.
- **Procedure:** Instrument client and server timings across 20 runs; report p50/p95 against targets.

3.2 SNFR2: Stateless server: no retained health data

- **Inputs:** Several completed analyses.
- **Expected Outcome:** No audio file, feature tensor, or result persists on the server after the response; repeat requests are independent.
- **Procedure:** After N analyses, inspect server storage and temp directories; confirm nothing remains; review request logs for absence of payloads.

3.3 SNFR3: Graceful handling of bad inputs

- **Inputs:** Corrupted file, non-cough audio, silence, 0.3 s clip, 5-minute clip, unsupported codec.
- **Expected Outcome:** Each returns a specific Inconclusive/error message without crash, hang, or misleading result; client shows [Try Again].
- **Procedure:** Script a malformed-input battery against the endpoint and UI; assert zero 5xx responses for user-caused cases and zero unhandled client exceptions.

3.4 SNFR4: Intuitive for first-time users

- **Inputs:** First-time tester with no instructions.
- **Expected Outcome:** Completes record → result without a tutorial or external help.
- **Procedure:** Moderated walkthrough with 3–5 first-time users; record completion rate, hesitation points, and missteps.

3.5 SNFR5: Modern browser compatibility

- **Inputs:** Chrome, Edge, Firefox, Safari (desktop + one mobile each).
- **Expected Outcome:** Full flow functional; recording, analysis, history, trends, and export behave equivalently.
- **Procedure:** Execute UCS1 on each browser in the matrix; log behavioral differences.

3.6 SNFR6: No health data in server logs

- **Inputs:** Analyses at all risk levels, including inconclusive results.
- **Expected Outcome:** Logs contain timings, trace IDs, error classes: no audio content, no risk levels tied to identifiers, no user-identifying health data.
- **Procedure:** Grep production-style logs after a test session for risk keywords, filenames, and payloads.

3.7 SNFR7: Accessibility (color + text, scaling)

- **Inputs:** Result screens rendered under high-contrast mode and enlarged browser font settings.
- **Expected Outcome:** Risk conveyed by color and text simultaneously; UI scales without clipping or overlap; usable at 200% zoom.
- **Procedure:** Enable Windows High Contrast and browser zoom on all result states; audit with a screen reader pass on the result text.

3.8 SNFR8: Modular architecture (ML layer independent of FastAPI)

- **Inputs:** ML pipeline codebase.
- **Expected Outcome:** The gate/segment/features/vote pipeline runs as a standalone Python module with no FastAPI imports; API layer is a thin wrapper.
- **Procedure:** Import and run the pipeline in a bare Python process on a reference clip; static-check imports for `fastapi` inside the ML package.

3.9 SNFR9: Swappable models and datasets

- **Inputs:** Alternative model checkpoint (`.pt/.onnx`) with identical head shape.
- **Expected Outcome:** Model swap completes by replacing the artifact + config; no application-layer code changes; smoke test passes.
- **Procedure:** Swap in a retrained checkpoint; run the golden-clip smoke test; confirm endpoints and UI unchanged.

3.10 SNFR10: Healthcare ethics compliance

- **Inputs:** All result states (Low/Medium/High/Inconclusive) in the shipped UI.
- **Expected Outcome:** Every state carries the pre-screening disclaimer; no wording implies diagnosis; confidence and vote data are displayed transparently.
- **Procedure:** Screenshot each state and audit against the ethics checklist; confirm disclaimers are non-dismissible and present in exported records.

4 Conclusion

This test plan details all functional and non-functional requirements of the COVID-19 Cough Detection web application, ensures verification through normal and unusual input cases, and validates compliance with privacy, usability, and performance constraints.

Functional requirements define the "what", objectives of specific features such as the guided three-cough capture, the cough-presence gate, deterministic vote aggregation, and local history. Each section defines normal and unusual inputs and expected outputs; inputs are user-triggered behaviors and follow procedural, step-by-step actions.

Non-functional requirements describe qualities of the system: constraints like latency, privacy, portability, and accessibility. The section defines how these qualities should behave; the conditions, timings, memory use, and system logs are measured by the system.